Федеральное агентство по рыболовству

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Астраханский государственный технический университет»

Система менеджмента качества в области образования, воспитания, науки и инноваций

ООО «ДКС РУС» по международному стандарту ISO 9001:2015

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОММУНИКАЦИЙ

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЗАЩИТА ИНФОРМАЦИИ

ОТЧЕТ

о выполнении лабораторной работы № 3

Разработка алгоритма хэш-функции

Вариант 1

Выполнил:
студент гр. ЗИНРб-51

_____ Генералов В. В.

«_____» _____ 2026 г.

Преподаватель:

_____ Савоськин И. А.

«_____» _____ 2026 г.

Астрахань – 2026

ВВЕДЕНИЕ

Хэш-функция отображает данные любой длины в короткий отпечаток фиксированной длины, что позволяет сравнивать и проверять данные без хранения оригинала. На хэш-функциях основаны хранение паролей, контроль целостности файлов и сообщений, электронная цифровая подпись (ЭЦП).

Цель работы: «освоить принципы построения хэш-функций, реализовать собственный алгоритм хэширования, провести его анализ и тестирование».

Задание по методическим указаниям состоит из трёх пунктов:

1. «Разработать алгоритм хэширования, основанный на некоторой функции согласно варианту (таблица 3.1). Алгоритм должен соблюдать свойство односторонности и выдавать хэш фиксированной длины, вне зависимости от входных данных».
2. «Разработать программный продукт, реализующий хэш-функцию, созданную на предыдущем шаге. Программа должна работать с произвольным количеством символов на входе (a-z, 0-9), включая пустое сообщение, а также по желанию пользователя добавлять к входному сообщению пользовательскую соль. Полученная хэш-сумма должна представляться в шестнадцатеричной системе счисления».
3. «Протестировать разработанную программу:».

- «Проверить эффект лавины (изменить один символ во входной последовательности и сравнить хэш-суммы)»;
- «Проверить на устойчивость к коллизиям, а также найти пару входных значений, имеющих одинаковый хэш»;
- «Проверить равномерность распределения хэш-сумм у различных входных данных (найти хэш для 100 сообщений и отсортировать, проверить, нет ли „скоплений“ в определенных диапазонах)».

Вариант 1 по таблице 3.1 методических указаний: основная функция sin(***m***).

В работе разработан итеративный алгоритм хэширования на основе функции sin с внутренним состоянием из 64 бит и выходом из 32 бит (восемь шестнадцатеричных цифр). Написанная на Python 3 программа реализует вычисление хэша с необязательной солью, проверку лавинного эффекта, поиск коллизии, оценку равномерности распределения и сравнение с наивной схемой. Выполнение пунктов задания подтверждено результатами тестирования: замена одного символа меняет в среднем 15,92 бита из 32, коллизия найдена в каждом из трёх запусков, скоплений хэшей по диапазонам нет.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Хэш-функция – функция, преобразующая массив входных данных произвольного размера в выходную битовую строку определённого (установленного) размера в соответствии с определённым алгоритмом. Хэш-код (хэш-сумма, хэш) – строка бит, являющаяся выходным результатом хэш-функции; далее n – длина хэш-кода в битах. Сообщение (входные данные) – строка бит произвольной конечной длины. Дополнение (соль) – приписывание дополнительных бит к строке бит; соль делает хэши одинаковых сообщений различными и обесценивает заранее вычисленные таблицы, но в отличие от ключа шифра не секретна и хранится вместе с хэшем.

Криптографическая хэш-функция – хэш-функция, удовлетворяющая следующим требованиям:

- сопротивление поиску прообраза: имея заданное значение h, должно быть сложно найти любое сообщение m, такое, что $h = \mathrm{hash}(m)$;
- сопротивление поиску второго прообраза: имея заданное входное значение m_1, должно быть сложно найти другое входное значение m_2, такое, что $\mathrm{hash}(m_1) = \mathrm{hash}(m_2)$;
- сопротивление коллизиям: должно быть сложно найти два различных сообщения m_1 и m_2 таких, что $\mathrm{hash}(m_1) = \mathrm{hash}(m_2)$.

Трудоёмкость поиска прообраза и второго прообраза – около 2^n вычислений, поиска коллизии – около $2^{n/2}$ по парадоксу дней рождения; при $n = 32$ это $2^{16} \approx 65$ тыс. попыток. Ожидаемое число сообщений до первой коллизии – $\sqrt{(\pi/2 \cdot 2^n)}$, для 32 бит около 82 137.

Практические хэш-функции строятся по итеративной схеме: состояние инициализируется константой, функция сжатия последовательно поглощает символы сообщения, финализация свёртывает состояние в выход фиксированной длины. Необратимость обеспечивают операции с потерей информации – свёртка широкого состояния в узкий выход, приведение по модулю 2^{32}, – а не основная функция варианта: sin на отрезке монотонности обратим через arcsin.

Эффект лавины – изменение одного символа входа меняет значительную часть бит выхода. Мерой служит расстояние Хэмминга d – число различающихся бит. Для идеальной функции d распределено по биномиальному закону $B(32; 0{,}5)$ с математическим ожиданием и средним квадратическим отклонением

$E[d] = 32 \cdot 0{,}5 = 16$, $\sigma = \sqrt{(32 \cdot 0{,}5 \cdot 0{,}5)} \approx 2{,}8$.

Равномерность проверяется подсчётом хэшей по диапазонам: при 100 хэшах и 16 диапазонах число хэшей в диапазоне подчиняется закону $B(100; 1/16)$ с ожиданием 6,25 и средним квадратическим отклонением 2,42; наибольшее из 16 значений обычно равно 10–12.

2 ОПИСАНИЕ АЛГОРИТМА И ПРОГРАММЫ

2.1 Постановка задачи

Методические указания не определяют ряд деталей, поэтому приняты допущения:

1. Аргумент синуса – не всё сообщение как число, а величина ***t*** из половины состояния и кода символа. Буквальный синус от числа-сообщения вырождается на 10 символах: мантисса double хранит 53 бита, и младшие символы перестают влиять на аргумент.
2. Длина выхода – 32 бита (восемь шестнадцатеричных цифр): коллизия находится перебором около 2^{16} сообщений, что делает требование «найти пару» выполнимым.
3. Внутреннее состояние – 64 бита (половины ***L*** и ***R***), шаг сжатия – раунд сети Фейстеля.
4. Значение синуса квантуется до целого $\mathrm{round}(\sin(\boldsymbol{t})\cdot 2^{30})$: реализации libm различаются в последнем бите double, и без квантования хэш зависел бы от платформы при любом таком расхождении, а с квантованием – лишь в редком случае, когда $\sin(\boldsymbol{t})\cdot 2^{30}$ близко к полуцелому.
5. Соль приписывается перед сообщением через разделитель `\x1f`. Алфавит не проверяется: символ берётся по коду, так как в лабораторной работе № 4 хэшируется текст ASCII (американский стандартный код для обмена информацией).
6. Критерии тестов: лавина – около 16 из 32 бит; коллизия – порядка 2^{16} попыток; скопление – свыше трёх ожиданий (6,25) в диапазоне.

2.2 Описание алгоритма

Алгоритм итеративный. Вход – соль, разделитель `\x1f` и сообщение (без соли – само сообщение). Состояние из половин ***L*** и ***R*** по 32 бита инициализируется константами ***IV_L*** и ***IV_R***, затем каждый символ с кодом ***c*** обрабатывается раундом сети Фейстеля $(\boldsymbol{L}, \boldsymbol{R}) \leftarrow (\boldsymbol{R}, \boldsymbol{L} \operatorname{xor} \boldsymbol{F}(\boldsymbol{R}, \boldsymbol{c}))$ с функцией раунда, построенной на основной функции варианта:

$$\boldsymbol{F}(\boldsymbol{R}, \boldsymbol{c}) = (\mathrm{round}(\sin(\boldsymbol{t})\cdot 2^{30}) \bmod 2^{32})\cdot \boldsymbol{P} \bmod 2^{32},$$

$$\boldsymbol{t} = (31\cdot \boldsymbol{R} + \boldsymbol{c}) \bmod 2^{32},$$

где

t – аргумент синуса;

P – простое число 16777619.

После обработки всех символов состояние свёртывается в 32 бита с подмешиванием длины входа ***N*** в символах и перемешивается финализатором fmix32 из MurmurHash3: $\boldsymbol{h} = \mathrm{fmix32}(\boldsymbol{L} \operatorname{xor} \boldsymbol{R} \operatorname{xor} \boldsymbol{N})$.

Раунд сети Фейстеля обратим при известном символе и любой ***F***: по половинам ***L′***, ***R′***

после раунда исходные восстанавливаются как $(\boldsymbol{L}, \boldsymbol{R}) = (\boldsymbol{R}' \text{ xor } \boldsymbol{F}(\boldsymbol{L}', \boldsymbol{c}), \boldsymbol{L}')$. Поэтому шаг сжатия переставляет 2^{64} состояний, а не отображает их случайно, и состояния разных сообщений не сходятся преждевременно. Односторонность создаёт не ***F*** и не умножение на нечётное ***P*** (обратимо по модулю 2^{32}), а свёртка ***L*** xor ***R***: 64 бита переходят в 32, каждому хэшу отвечает около 2^{32} состояний, прообраз ищется только перебором.

Длина в финализации отделяет входы разной длины с совпавшим состоянием (усиление Меркла – Дамгора). Последовательность вычислений функции ***my_hash*** в виде псевдокода показана на рисунке 2.1, параметры алгоритма приведены в таблице 2.1.

```
my_hash(message, salt):
    data ← salt + "\x1f" + message, если salt непуста, иначе message
    (L, R) ← (IV_L, IV_R)
    для каждого символа ch в data:
        c ← ord(ch)
        t ← (31·R + c) mod 2^32
        F ← (round(sin(t)·2^30) mod 2^32)·P mod 2^32
        (L, R) ← (R, L xor F)
    вернуть fmix32(L xor R xor len(data))
```

Рисунок 2.1 – Псевдокод функции my_hash

Таблица 2.1 – Параметры алгоритма

Параметр	Значение	Обоснование
IV_L, ***IV_R***	0x811C9DC5, 0x9E3779B9	Смещение FNV-1a (хэш Фаулера – Нолла – Во) и целая часть $2^{32}/\varphi$ (φ – золотое сечение): ненулевое начальное состояние с различными половинами, ненулевой хэш пустой строки
P	16777619	Простое число FNV-1a, нечётное: умножение обратимо по модулю 2^{32}, распределяет квантованный синус по всем битам
Множитель ***MUL***	31	Нечётный (как в hashCode Java): $\boldsymbol{t} = 31 \cdot \boldsymbol{R} + \boldsymbol{c}$ обратимо по ***R***, символ связан с состоянием
Масштаб квантования ***SIN_SCALE***	2^{30}	round(sin(***t***)·2^{30}) – целое: расхождение libm в последнем бите double меняет его, только если sin(***t***)·2^{30} близко к полуцелому, то есть крайне редко
Константы fmix32 ***FMIX_1***, ***FMIX_2***	0x85EBCA6B, 0xC2B2AE35; сдвиги 16, 13, 16	Финализатор MurmurHash3 (xor-сдвиги и умножения по модулю 2^{32}): лавина по всем 32 битам, обратим, информацию не теряет
Разделитель соли ***SEPARATOR***	`\x1f`	Не входит в алфавит a–z, 0–9: hash(«abc», без соли) ≠ hash(«bc», соль «a»)

2.3 Описание программы

Программа написана на языке Python 3 с использованием только стандартной библиотеки и запускается из консоли без аргументов. Она состоит из одного модуля, в котором собраны хэш-функция на основе sin(***m***), тесты из задания и меню режимов.

Ядро составляют функции ***round_function*** (функция раунда ***F***), ***compress*** (раунд Фейстеля), ***fmix*** (финализатор), ***my_hash*** (хэш с необязательной солью), ***to_hex*** (запись восемью шестнадцатеричными цифрами) и ***naive_hash*** (наивная схема). Тестам служат ***hamming***, ***random_message***, ***sample_messages***, ***avalanche_pair***, ***avalanche_stats***, ***find_collision***, ***distribution***,

structural_checks; режимам меню – ***mode_hash***, ***mode_avalanche***, ***mode_collision***, ***mode_distribution***, ***mode_compare***; цикл меню – ***main***.

Меню, показанное на рисунке 3.1, содержит режимы «1» – «5»: хэш сообщения с солью по желанию, эффект лавины, структурные проверки и поиск коллизии, распределение 100 хэшей по 16 диапазонам, сравнение с наивной схемой; пункт «0» завершает работу.

Вход – строка любой длины, включая пустую (клавиша «Enter»), и необязательная соль. Выход – ровно восемь шестнадцатеричных цифр с ведущими нулями. Тестовые сообщения порождает генератор с фиксированным начальным значением: символы a–z, 0–9, длина от 1 до 12 (в статистике лавины – от 2 до 12). На неверный пункт меню программа отвечает «Нет такого пункта меню.»; в режиме 2 пустое сообщение, позиция вне сообщения и символ, совпадающий с исходным, отклоняются с повторным запросом; в режиме 3 при нецелом начальном значении генератора программа выводит «Нужно целое число; выполняются три запуска с seed 2026, 1, 2.» и продолжает с тремя запусками по умолчанию. Сочетание «Ctrl+C» или «Ctrl+D» завершает программу с сообщением «Выход.».

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.1 Вычисление хэша

Пункт 2 задания и требование фиксированной длины хэша из пункта 1 проверялись в режиме 1 (***mode_hash***). Меню программы и хэши сообщения, сообщения с солью и пустого сообщения показаны на рисунке 3.1, хэши сообщений длиной от 0 до 10 000 символов приведены в таблице 3.1.

```
Лабораторная работа №3. Хэш-функция на основе sin(m) (вариант 1), выход 32 бита
  1 — вычислить хэш сообщения (с солью по желанию)
  2 — эффект лавины: замена одного символа
  3 — устойчивость к коллизиям и поиск пары с одинаковым хэшем
  4 — равномерность распределения: 100 сообщений по 16 диапазонам
  5 — сравнение с наивной схемой sin(сумма кодов)
  0 — выход
Выбор: 1
  Сообщение (Enter — пустое): abc
  Добавить соль? (y/n): n
  hash('abc') = 7139aeec

Выбор: 1
  Сообщение (Enter — пустое): abc
  Добавить соль? (y/n): y
  Соль: s1
  hash('abc', соль 's1') = c75fe06f

Выбор: 1
  Сообщение (Enter — пустое):
  Добавить соль? (y/n): n
  hash('') = 4a216c31
```

Рисунок 3.1 – Режим 1: хэш сообщения, хэш с солью, хэш пустого сообщения

Таблица 3.1 – Проверка фиксированной длины, пустого ввода и соли

Сообщение	Соль	Число символов входа	Хэш	Число цифр хэша	Сообщение	Соль	Число символов входа	Хэш	Число цифр хэша
Пустая строка	–	0	4a216c31	8	abc	s1	6	c75fe06f	8
Пустая строка	s1	3	3024ce6b	8	abc	s2	6	02d1a586	8
a	–	1	51e5e29f	8	uuuuuu… (100 символов)	–	100	cd855fef	8
ab	–	2	5ab98633	8	oooooo… (10 000 символов)	–	10 000	6099dbde	8
ba	–	2	7f548c36	8	A!	–	2	55f32b24	8
abc	–	3	7139aeec	8	–	–	–	–	–

Хэш любого входа, от пустого до 10 000 символов, состоит ровно из восьми шестнадцатеричных цифр; соль меняет хэш того же сообщения.

3.2 Эффект лавины

Эффект лавины (первая проверка пункта 3 задания) исследовался в режиме 2 (***mode_avalanche***): программа заменяет один символ сообщения, выводит оба хэша в шестнадцатеричном и двоичном виде, отмечает различающиеся биты и вычисляет расстояние

Хэмминга, затем приводит статистику по 1000 случайным парам сообщений длиной от 2 до 12 символов, различающихся одним символом (***avalanche_stats***). Здесь и далее меню на экранах опущено.

Работа режима показана на рисунке 3.2, результаты замены одного символа в разных позициях приведены в таблице 3.2.

```
Выбор: 2
  Сообщение (не короче 1 символа): hello
  Позиция заменяемого символа (0..4): 2
  Новый символ: x
  'hello'          → 321a9e7f  00110010000110101001111001111111
  'hexlo'          → 02c6565d  00000010110001100101011001011101
  различаются биты:              ^^    ^^ ^^^  ^^  ^     ^   ^
  расстояние Хэмминга: 12 из 32 (38%); ожидание для идеальной функции — 16
  Статистика по 1000 случайным парам: среднее 15.92, минимум 6, максимум 26 из 32 (биномиальное
      распределение B(32, 1/2): σ ≈ 2.8)
```

Рисунок 3.2 – Режим 2: эффект лавины

Таблица 3.2 – Эффект лавины при замене одного символа

Исходное сообщение	Изменённое сообщение	Хэш до замены	Хэш после замены	Расстояние Хэмминга, бит	Доля изменённых бит, %
hello	jello (позиция 0: h→j)	321a9e7f	fd5a398e	17	53
hello	hexlo (позиция 2: l→x)	321a9e7f	02c6565d	12	38
hello	hellp (позиция 4: o→p)	321a9e7f	c9d74042	23	72
abcdefghij12	abcdefghij13 (позиция 11: 2→3)	220b77e8	27ca3ab7	15	47
password1	password2 (позиция 8: 1→2)	cc431475	82c6c6be	16	50
a	b (позиция 0: a→b)	51e5e29f	42c049d2	15	47

По отдельным парам расстояние составляет 12–23 бита; замена последнего символа в строке abcdefghij12 из 12 символов меняет 15 бит, то есть вырождения на длинных входах нет. По 1000 случайным парам среднее расстояние равно 15,92 бита (49,7 %) при ожидании 16 = 32/2, минимум 6, максимум 26. Разброс 6–26 не дефект: при законе $\boldsymbol{B}(32; 0{,}5)$ с $\boldsymbol{\sigma} \approx 2{,}8$ значения 8–24 бита ($\pm 3\boldsymbol{\sigma}$) типичны, единичные 6 и 26 допустимы.

3.3 Устойчивость к коллизиям

Вторая проверка пункта 3 задания – устойчивость к коллизиям и поиск пары с одинаковым хэшем – выполнялась в режиме 3 (***mode_collision***). Сначала программа хэширует структурные пары, на которых наивные схемы дают коллизии: перестановку символов, повтор символа против пустого входа, перенос символа из сообщения в соль и ведущие нули. Затем ищется коллизия методом дней рождения (***find_collision***): различные случайные сообщения из символов a–z, 0–9 длиной от 1 до 12 хэшируются до первого повтора хэша; при пустом вводе начального значения генератора выполняются три запуска, при вводе целого числа – один.

Три запуска с начальными значениями генератора 2026, 1 и 2 показаны на рисунке 3.3.

```
Выбор: 3
  Структурные проверки (пары, на которых ломаются наивные схемы):
    hash('ab')                = 5ab98633   hash('ba')                = 7f548c36   различны
    hash('aa')                = 2c7d5d6b   hash('')                  = 4a216c31   различны
    hash('abc')               = 7139aeec   hash('bc', соль 'a')      = d065488d   различны
    hash('0')                 = 98c1e4ce   hash('00')                = 768132b1   различны
    hash('a')                 = 51e5e29f   hash('a')                 = 51e5e29f   СОВПАЛИ
  Seed генератора сообщений (Enter — три запуска с seed 2026, 1, 2):
  Поиск коллизии методом дней рождения (различные сообщения из a-z0-9 длины 1..12);
  ожидание для 32 бит: sqrt(π/2 · 2^32) ≈ 82137 сообщений, разброс ±50 % (σ ≈ 0.52·E)
    seed  2026: 'dmr75y' и '69ny4tv5t8k9' → cda8d68e; различных сообщений 89364, время 0.49 с
    seed     1: 'yqq7aquw' и 'qaqkg45yo94x' → f652e063; различных сообщений 146559, время 0.82 с
    seed     2: 'c8psmht8jw96' и 'xh9' → 8c73b638; различных сообщений 76699, время 0.42 с
  Среднее по 3 запускам: 104207 (теория 82137)
```

Рисунок 3.3 – Режим 3: структурные проверки и поиск коллизии

Все четыре структурные пары у итоговой схемы дают различные хэши, контрольная пара одинаковых входов – одинаковые; наивная схема sin от суммы кодов совпадает только на паре «ab» и «ba». Результаты поиска коллизии приведены в таблице 3.3.

Таблица 3.3 – Поиск коллизии методом дней рождения

Начальное значение генератора	Число различных сообщений до коллизии	Сообщение 1	Сообщение 2	Общий хэш	Время, с
2026	89 364	dmr75y	69ny4tv5t8k9	cda8d68e	0,49
1	146 559	yqq7aquw	qaqkg45yo94x	f652e063	0,82
2	76 699	c8psmht8jw96	xh9	8c73b638	0,42
Ожидание $\sqrt{(\pi/2 \cdot 2^{32})}$	82 137	–	–	–	–
Среднее по трём запускам	104 207	–	–	–	–

Коллизия найдена в каждом из трёх запусков, каждый запуск длится меньше 1 с. Число сообщений до совпадения – 76 699, 89 364 и 146 559, среднее 104 207 лежит в пределах одного среднего квадратического отклонения от ожидания 82 137: разброс ±50 % присущ парадоксу дней рождения, функция ведёт себя как случайная.

Время поиска позволяет оценить и односторонность. Обратной формулы нет: свёртка $\boldsymbol{L}$ xor $\boldsymbol{R}$ 64-битного состояния в 32 бита теряет половину информации. Прообраз находится перебором 2^{32} сообщений; при скорости около 180 тыс. хэшей в секунду (см. таблицу 3.3) это порядка 7 часов. Односторонность обеспечена на уровне формулы, не криптографически.

3.4 Равномерность распределения

Третья проверка пункта 3 задания – равномерность распределения хэш-сумм – выполнялась в режиме 4 (***mode_distribution***): хэши 100 различных случайных сообщений сортируются и подсчитываются по 16 диапазонам первой шестнадцатеричной цифры. Полный отсортированный список хэшей приведён в приложении 1 (таблица П1.1).

Работа режима показана на рисунке 3.4 (список хэшей сокращён).

```
Выбор: 4
  Отсортированные хэши 100 различных случайных сообщений (seed 2026):
    00b0324f vhur2          0bbcff1e v0vg587n1wvd   0cd120d0 c5ib4          0d8497f1 j2i1
    0e5ce98f 9hr            1109a110 9vu5zulm3      114fa40b t              118d05ac g9
    …
    f65b5f06 iyma41         f9d96fe1 j7myagy        fa07df57 t964z1msrac    fa697c8d w3wf
  Распределение по 16 диапазонам (первая hex-цифра), ожидание 6.25 на диапазон, σ ≈ 2.42:
    0xxxxxxx     5  #####
    1xxxxxxx    12  ############
    2xxxxxxx    10  ##########
    3xxxxxxx     6  ######
    4xxxxxxx     5  #####
    5xxxxxxx     8  ########
    6xxxxxxx     3  ###
    7xxxxxxx     5  #####
    8xxxxxxx     5  #####
    9xxxxxxx     8  ########
    axxxxxxx     5  #####
    bxxxxxxx     6  ######
    cxxxxxxx     4  ####
    dxxxxxxx     5  #####
    exxxxxxx     7  #######
    fxxxxxxx     6  ######
  Минимум 3, максимум 12; максимум из 16 диапазонов при равномерном распределении обычно 10–12,
      скоплением считается 19 и более (свыше трёх ожиданий).
  Скоплений нет.
```

Рисунок 3.4 – Режим 4: равномерность распределения

Все 100 хэшей различны. На диапазон приходится от 3 (6xxxxxxx) до 12 (1xxxxxxx) хэшей, отклонение от ожидания 6,25 – от −3,25 до +5,75. Максимум 12 превышает ожидание на 2,4 средних квадратических отклонения (2,42), что нормально для наибольшей из 16 величин; скоплений нет.

3.5 Анализ результатов

Результаты тестов соответствуют поведению случайной функции: средняя лавина близка к 16 битам, число сообщений до коллизии – к ожиданию по парадоксу дней рождения, частоты по диапазонам – к биномиальному закону.

В режиме 5 (***mode_compare***) итоговая схема сопоставлена на тех же сообщениях с наивной схемой floor(|sin(сумма кодов)|·2^{32}), результаты сравнения приведены в таблице 3.4.

Таблица 3.4 – Сравнение наивной и итоговой схем

Схема	Средняя лавина, бит	Разброс лавины, бит	Максимум хэшей в диапазоне	Совпадение хэшей «ab» и «ba»
Наивная	15,8	6–25	21	Да
Итоговая	15,9	6–26	12	Нет

Наивная схема при сходной лавине коммутативна и даёт скопление из-за арксинусного распределения |sin|. Цепочка состояния в ***compress*** устраняет коммутативность, умножение на ***P*** и финализатор ***fmix*** – скопление.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Где используется хэширование и для чего оно нужно?

Хэширование даёт короткий отпечаток данных фиксированной длины, по которому данные сравнивают и проверяют, не храня и не раскрывая оригинал. Основные применения: хранение паролей в виде хэша с солью вместо открытого текста; контроль целостности файлов и сообщений; ЭЦП, где подписывается хэш документа, а не сам документ; хэш-таблицы для быстрого поиска; дедупликация и идентификация данных по отпечатку.

Вопрос 2. Какие свойства должна иметь хэш-функция?

Детерминированность и фиксированная длина выхода (см. таблицу 3.1); три вида стойкости – сопротивление поиску прообраза, второго прообраза и коллизий; лавинный эффект (см. таблицу 3.2); равномерность распределения (см. рис. 3.4); вычислительная эффективность. Трудоёмкость поиска прообраза для n-битного хэша составляет 2^n, коллизии – около $2^{n/2}$; для 32-битного хэша это 2^{16}, что по порядку величины подтверждает таблица 3.3, а рисунок 3.3 показывает отсутствие тривиальных коллизий на структурных парах.

Вопрос 3. Чем отличается шифрование от хэширования?

Шифрование обратимо: обладатель ключа восстанавливает сообщение; хэширование необратимо по построению. Длина шифртекста сопоставима с длиной сообщения, длина хэша фиксирована – это сжатие с потерей информации. Цель шифрования – конфиденциальность, цель хэширования – целостность и аутентичность. Ключ шифра секретен, соль хэш-функции – нет. Механизмы дополняют друг друга: в ЭЦП асимметричный шифр применяется не к сообщению, а к его хэшу.

Вопрос 4. Что такое эффект лавины и коллизия, и чем она может быть опасна?

Эффект лавины – изменение одного символа или бита входа меняет около половины бит выхода; мера – расстояние Хэмминга (в среднем 15,92 из 32 бит, см. рис. 3.2). Коллизия – два различных входа с одинаковым хэшем; для функции с конечным выходом она неизбежна по принципу Дирихле, вопрос лишь в трудоёмкости поиска. Опасность: подмена подписанного документа с сохранением подписи, вход по «чужому» паролю с тем же хэшем, атака дня рождения – заготовка двух версий документа с одним хэшем. Пример: сообщения «dmr75y» и «69ny4tv5t8k9» дают одинаковый хэш cda8d68e (см. таблицу 3.3).

ЗАКЛЮЧЕНИЕ

Разработан алгоритм хэширования на основе функции sin(***m***) (вариант 1): 64-битное состояние из половин ***L*** и ***R***, раунд сети Фейстеля на каждый символ, 32-битный выход из восьми шестнадцатеричных цифр. Программа на языке Python 3 реализует пять режимов: хэширование с необязательной солью, лавинный тест, поиск коллизии, проверку равномерности и сравнение с наивной схемой.

Пункты 1–3 задания выполнены. Функция односторонняя на уровне формулы: свёртка состояния из 64 бит в 32 бита оставляет для поиска прообраза только перебор около 2^{32} сообщений, порядка 7 часов (см. подраздел 3.3). Длина выхода фиксирована для любого входа, включая пустой, соль добавляется по желанию, хэш выводится восемью шестнадцатеричными цифрами (см. рис. 3.1 и таблицу 3.1). Замена одного символа меняет в среднем 15,92 бита из 32 (см. рис. 3.2 и таблицу 3.2). Коллизия найдена за 77–147 тыс. сообщений при ожидании 82 тыс. (см. рис. 3.3 и таблицу 3.3). Наибольшее число хэшей в диапазоне – 12 из 100 при ожидании 6,25 (см. рис. 3.4 и таблицу П1.1). Наивная схема sin от суммы кодов при сходной лавине коммутативна и даёт скопление до 21 хэша в диапазоне, итоговая схема этих дефектов не имеет (см. таблицу 3.4).

Найденная коллизия – свойство выбранной 32-битной длины, а не дефект схемы; промышленные MD5 (Message Digest 5, 128 бит) и SHA-256 (Secure Hash Algorithm, 256 бит) используют десятки раундов целочисленной арифметики.

Ограничения: 32-битный выход, один раунд на символ, зависимость от реализации ***math.sin*** ослаблена квантованием до 2^{30}. В лабораторной работе № 4 подписывается сам хэш, а при проверке подписи он сравнивается с результатом её расшифрования открытым ключом RSA (Ривеста – Шамира – Адлемана). Этот результат всегда меньше модуля, поэтому модуль должен превышать 2^{32}, иначе проверка не сойдётся для хэшей, не меньших модуля; коллизия самого хэша при этом по-прежнему даёт одну подпись на два сообщения.

ПРИЛОЖЕНИЕ 1

ОТСОРТИРОВАННЫЕ ХЭШИ 100 СООБЩЕНИЙ

Хэши 100 случайных сообщений из режима 4, отсортированные по возрастанию, приведены в таблице П1.1; первая цифра хэша определяет его диапазон (см. рис. 3.4).

Таблица П1.1 – Хэши 100 случайных сообщений

Сообщение	Хэш	Сообщение	Хэш	Сообщение	Хэш	Сообщение	Хэш
vhur2	00b0324f	lar9qiv	2c767d48	j8rov	765fb3c4	u5	b9309b49
v0vg587n1wvd	0bbcff1e	gb8ntczy87k6	2dc7d161	mq5365	76a06ec6	lg6ure	bda3f70e
c5ib4	0cd120d0	q3mxr6	31d3b69a	jv	7ac5c621	iw0yyp0oil	be969995
j2i1	0d8497f1	0clftx8qi	336d4d84	3	7b16b32b	0k	c1552a16
9hr	0e5ce98f	v7uda	38c839a0	3nkt	80bd6087	n8jpi	c15882e8
9vu5zulm3	1109a110	6nr	3abb46c3	0juj6c4a	80e54c1c	bi7n7ikl	c91f1241
t	114fa40b	ztnd0e68t4f6	3d101ec3	5z7q0467bo	847a38a6	vvwu4q5u	caf524de
g9	118d05ac	ht0ub	3eeab6d9	ukqqe	899f34df	qoku1	d5f23f43
c8rwjau5i8	125a3902	5i9uxz88mj9j	423dd2fd	d7g	8a5ebcfc	97	da9283ee
1fz95h16	12e42b66	3v	44365d4e	044vacohru	9183b228	n167	dc386117
kldlj058	156f7c40	8k7blu	49167941	6rxf9m8a	921fb1bf	6q2l14er5xgm	dddbea64
0j2afa28fmph	184ca16e	fj	4b9d8d89	9igbr0cl96o	94c2ae20	thw45	dfedf739
b89zqjobb	1883ff89	e9zcmlc8d2m	4ffdf038	wbb64ut2uh5	95cbdb7f	jkbh6axex	e1293216
yv9qtcmkss8s	1be07691	mxw339	517da59e	r180fyt	981d2d66	fhsg2a5unz	e2639b60
77ygn	1f13374e	x6qyswd1bsv	546d4eef	0	98c1e4ce	u5uvxle9l	e455abe2
7wt	1f1e5ec6	9csu3g2	58d55b0e	fl9	9c6af06b	wlq	e5195f08
07	1fa38bae	50s3lu74	5cabf473	vm9dhkhhc	9d1aff54	g1kgd	e8514b57
jidcrtlir8wi	2160e5cc	8nhr9nsr9ig	5cc1da0c	go90952pa	a1933b87	8lbwkw9	ea2e29ec
375	22d2a380	g	5ded5cf0	0adgp	a46984f0	f5	ea513cc5
t60	259aed56	u6	5ef0728c	ey	a53c8e44	p4do6jzxo9r	f0bae600
vf98ss3jtbxx	2604fb15	3zzb9	5fae221f	kj7n1v3o	aba95af5	ww0y	f3452c4b
wu3czun31h	271b0462	tmk0j5aw	60c3d430	1	aefec0f6	iyma41	f65b5f06
dw	28fb85c2	h77rgnoa	69c8b189	n	b1b315b5	j7myagy	f9d96fe1
wwy6e	2acaf4de	6x8f	69fe85e2	6	b4fac1e7	t964z1msrac	fa07df57
4i6s	2c2b763d	14j4oh348q9z	72f81227	840qu	b840dc30	w3wf	fa697c8d